

Complete Architecture Blueprint

Phase 1: Agency-Level System Architecture

Document Type	Architecture Blueprint & Technical Specification
Project	Beauty Care by Nabila Lahore - Luxury Salon Platform
Version	1.0
Classification	Confidential
Framework	Next.js 16 / TypeScript / Tailwind CSS 4
Date	May 2026

1. Executive Summary & Vision

Beauty Care by Nabila Lahore is not merely a salon website; it is a **digital luxury experience platform** designed to rival the online presence of international luxury beauty brands such as Dior Beauty, Sephora Luxury, and premium Dubai salon brands. The platform must evoke an emotional premium response within the first three seconds of engagement, establishing trust, desire, and exclusivity through cinematic visual storytelling, ultra-smooth micro-interactions, and a conversion-optimized user journey that transforms casual visitors into loyal clientele.

This architecture blueprint defines every structural, technical, and experiential decision required to build a world-class luxury salon platform. It covers system architecture, component hierarchies, animation strategies, responsive design systems, state management, Firebase backend integration, Sanity CMS content modeling, deployment pipelines, and a phased development roadmap. Every decision is made with the singular goal of delivering an experience that feels emotionally premium, cinematically rich, and conversion-optimized at every touchpoint.

1.1 Design Philosophy

The design philosophy is rooted in the concept of **controlled opulence** - the art of communicating luxury through restraint rather than excess. Drawing from Apple's minimalist precision, Dior's typographic elegance, and Sephora's conversion mastery, the platform uses generous whitespace, champagne gold accents on matte black surfaces, and cinematic motion design to create an atmosphere that feels like stepping into a luxury boutique. Every pixel, every transition, and every interaction must reinforce the perception that this is not just a salon - it is a destination for transformation.

1.2 Brand Identity System

Element	Specification	Rationale
Primary Background	Matte Black (#0C0C10)	Conveys luxury, depth, and sophistication
Primary Accent	Champagne Gold (#D4AF37)	Signals premium quality and exclusivity
Secondary Background	Ivory (#F8F5F0)	Soft contrast for readability and warmth
Tertiary Accent	Warm Beige (#DCC7AA)	Bridges gold and ivory for harmony
Heading Font	Playfair Display	Editorial luxury, high-fashion association
Body Serif	Cormorant Garamond	Elegant readability, timeless sophistication
Body Sans	Inter	Clean UI text, legibility at small sizes

Table 1: Brand Identity Specification

1.3 Experience Principles

The platform operates under five immutable experience principles that govern every design and engineering decision. First, **Cinematic First Impression**: the hero section must deliver a full-screen, video-driven, emotionally resonant experience that communicates the brand within 3 seconds. Second, **Tactile Interaction**: every tap, swipe, and hover must produce a perceptible micro-animation that makes the interface feel alive and responsive. Third, **Effortless Conversion**: the path from inspiration to booking must be achievable in under 60 seconds with zero friction. Fourth, **Content-Driven Luxury**: every image, testimonial, and service description must meet editorial-quality standards, never stock-quality. Fifth, **Mobile-Native**: the mobile experience is not a responsive adaptation; it is the primary experience, designed first and enhanced for desktop.

2. Complete System Architecture

The system architecture follows a **headless, composable architecture pattern** that decouples the presentation layer from content management and backend services. This approach enables independent scaling of frontend and backend components, allows content editors to work without developer intervention, and supports multi-channel content delivery in the future. The architecture is designed to handle high-traffic bridal season spikes while maintaining sub-2-second page load times on 4G connections.

2.1 High-Level Architecture Layers

Layer	Technology	Purpose
Presentation	Next.js 16 + App Router	SSR/SSG pages, RSC, client islands
Styling	Tailwind CSS 4 + Framer Motion	Utility-first styling + animation primitives
Motion	GSAP ScrollTrigger	Cinematic scroll-driven animations
State (Client)	Zustand	Lightweight global state (UI, booking flow)
State (Server)	TanStack Query	Server state caching, optimistic updates
CMS	Sanity Studio	Content authoring, real-time previews
Database	Prisma + SQLite (primary), Firebase (real-time)	Structured data + real-time availability
Auth	NextAuth.js v4	Admin authentication, session management
Media	Cloudinary	Image optimization, transformations, CDN
Hosting	Vercel	Edge deployment, ISR, serverless functions

Analytics	Vercel Analytics + PostHog	Performance + user behavior tracking
-----------	----------------------------	--------------------------------------

Table 2: Technology Stack Architecture

2.2 Data Flow Architecture

The data flow follows a unidirectional pattern optimized for performance and developer experience. Static content (services, about, testimonials) is fetched at build time via Sanity GROQ queries during Next.js Static Site Generation, producing pre-rendered HTML that loads instantly. Dynamic content (booking availability, VIP membership status) flows through TanStack Query on the client, with background refetching ensuring data freshness without blocking the UI. Real-time booking availability uses Firebase Realtime Database to synchronize appointment slots across concurrent users, preventing double-booking during high-traffic bridal season periods. Media assets follow a Cloudinary transformation pipeline: original uploads are stored in Cloudinary, and responsive variants with automatic format negotiation (WebP/AVIF) are generated on-the-fly via URL parameters, with aggressive CDN caching at the edge.

2.3 Rendering Strategy

Page	Strategy	Revalidation	Rationale
Homepage	SSG + ISR	Every 60s	Fast initial load, fresh content hourly
Bridal Studio	SSG + ISR	Every 120s	Showcase content, infrequent updates
Services	SSG + ISR	Every 300s	Service catalog rarely changes
Gallery	SSG + ISR	Every 180s	Portfolio updates, moderate freshness
Booking	SSR + Client	No cache	Real-time availability required
About	SSG	On deploy	Static content, build-time only
Contact	SSG	On deploy	Static information page
VIP Membership	SSG + Client	Every 300s	Benefits static, status dynamic
Admin Dashboard	CSR	No cache	Fully client-rendered, authenticated

Table 3: Rendering Strategy per Page

3. Folder Structure

The folder structure follows Next.js 16 App Router conventions with domain-driven organization. Each route is self-contained with its own components, hooks, and server actions. Shared components are

organized by functional domain (layout, ui, sections) rather than by technical type, which reduces import depth and improves code discoverability. The structure is designed to scale from a single developer to a team of 5-8 without requiring reorganization.

src/	
app/	
layout.tsx	# Root layout: fonts, theme, providers
page.tsx	# Homepage
globals.css	# Global styles, CSS variables
bridal/	
page.tsx	# Bridal Studio page
services/	
page.tsx	# Services catalog
gallery/	
page.tsx	# Instagram-style gallery
booking/	
page.tsx	# Online booking system
about/	
page.tsx	# About Nabila
contact/	
page.tsx	# Contact & WhatsApp integration
vip/	
page.tsx	# VIP Membership
admin/	
layout.tsx	# Admin shell with sidebar
page.tsx	# Dashboard overview
bookings/	
page.tsx	# Booking management
services/	
page.tsx	# Service CRUD
gallery/	
page.tsx	# Gallery management
testimonials/	
page.tsx	# Testimonial moderation
content/	
page.tsx	# CMS content editor
api/	
booking/	
route.ts	# Booking API endpoints
contact/	
route.ts	# Contact form handler
webhook/	

sanity/
route.ts # Sanity webhook handler
components/
layout/
Header.tsx # Animated header with scroll behavior
Footer.tsx # Luxury footer
Navigation.tsx # Desktop & mobile navigation
MobileMenu.tsx # Full-screen mobile menu overlay
PageTransition.tsx # Framer Motion page transitions
sections/
HeroSection.tsx # Cinematic hero with video/image
BridalShowcase.tsx # Bridal portfolio carousel
ServicesGrid.tsx # Animated service cards
GalleryMasonry.tsx # Instagram-style masonry grid
TestimonialsCarousel.tsx # Luxury testimonial carousel
CTABanner.tsx # Conversion call-to-action
VIPSection.tsx # VIP membership teaser
booking/
BookingWizard.tsx # Multi-step booking flow
ServiceSelector.tsx # Service category selection
StylistSelector.tsx # Stylist preference
DateTimePicker.tsx # Calendar + time slot picker
BookingConfirmation.tsx # Confirmation with WhatsApp link
ui/
LuxuryButton.tsx # Gold-bordered animated button
GoldDivider.tsx # Champagne gold separator
AnimatedCounter.tsx # Number counter animation
ParallaxImage.tsx # Scroll-driven parallax
RevealOnScroll.tsx # Scroll-triggered reveal wrapper
VideoHero.tsx # Autoplay video with overlay
MagneticButton.tsx # Mouse-following magnetic effect
CursorGlow.tsx # Custom cursor with gold glow
ShimmerLoader.tsx # Gold shimmer skeleton
LuxuryCard.tsx # Hover-animated service card
admin/
DashboardShell.tsx # Admin layout with sidebar
BookingCalendar.tsx # Calendar view for bookings
StatsCards.tsx # Revenue & booking KPIs
ContentEditor.tsx # Rich text editor bridge
hooks/
useScrollProgress.ts # Scroll position 0-1 normalized
useInView.ts # Intersection Observer with threshold

useMediaQuery.ts	# Responsive breakpoint detection
useBookingFlow.ts	# Booking state machine
useGSAP.ts	# GSAP context cleanup
useCursorGlow.ts	# Custom cursor position tracking
lib/	
sanity/	
client.ts	# Sanity client configuration
queries.ts	# GROQ query definitions
schemas/	# Sanity schema definitions
service.ts	
testimonial.ts	
gallery.ts	
bridal.ts	
vipTier.ts	
siteSettings.ts	
firebase/	
config.ts	# Firebase app initialization
booking.ts	# Real-time booking operations
availability.ts	# Slot availability sync
cloudinary/	
config.ts	# Cloudinary URL builder
transforms.ts	# Image transformation presets
utils.ts	# Shared utility functions
constants.ts	# Brand constants, breakpoints
stores/	
bookingStore.ts	# Zustand: booking flow state
uiStore.ts	# Zustand: UI state (menu, modals)
types/	
sanity.ts	# Sanity document type definitions
booking.ts	# Booking & availability types
service.ts	# Service catalog types
index.ts	# Re-exports
styles/	
fonts.ts	# Next.js font configuration

Figure 1: Complete Project Folder Structure

4. Routing Plan

The routing plan leverages Next.js 16 App Router with file-system based routing. Each route is a self-contained directory with its own page.tsx and optional layout.tsx. The admin routes share a common

layout with authentication guards and a sidebar navigation shell. Public routes use the root layout with the animated header, footer, and page transition wrapper. Dynamic segments are used for individual service detail pages and gallery item views, with generateStaticParams pre-rendering all known routes at build time for instant navigation.

4.1 Public Routes

Route	Page	Key Features
/	Homepage	Cinematic hero, bridal teaser, services preview, testimonials, CTA
/bridal	Bridal Studio	Full bridal portfolio, process timeline, inquiry form
/services	Services	Categorized service grid, pricing, duration, book-now CTAs
/services/[slug]	Service Detail	Full description, gallery, related services, booking link
/gallery	Gallery	Masonry grid, category filters, lightbox viewer, lazy-load
/booking	Booking	Multi-step wizard, real-time availability, WhatsApp fallback
/about	About	Nabila story, team, awards, philosophy, press mentions
/contact	Contact	Map, WhatsApp chat, form, social links, hours
/vip	VIP Membership	Tiers, benefits comparison, enrollment CTA, exclusivity messaging

Table 4: Public Route Definitions

4.2 Admin Routes

Route	Page	Access Level
/admin	Dashboard	Authenticated admin only
/admin/bookings	Booking Management	View, confirm, cancel, reschedule bookings
/admin/services	Service Management	CRUD operations for service catalog
/admin/gallery	Gallery Management	Upload, organize, tag gallery images
/admin/testimonials	Testimonial Moderation	Approve, edit, feature testimonials
/admin/content	Content Editor	Bridge to Sanity Studio for page content

Table 5: Admin Route Definitions

4.3 API Routes

API routes follow RESTful conventions and serve as the server-side layer for operations that require authentication validation, database writes, or third-party integrations. The booking API handles appointment creation with optimistic locking via Firebase transactions to prevent double-booking. The contact API processes form submissions and triggers WhatsApp notification webhooks. The Sanity webhook route receives content update notifications and triggers on-demand ISR revalidation for affected pages, ensuring content changes propagate within seconds.

5. Component Hierarchy

The component hierarchy is organized into four distinct layers: Layout Components (persistent across routes), Section Components (page-level building blocks), Interactive Components (user-driven functionality), and Primitive UI Components (reusable design tokens as components). This separation ensures that layout concerns are isolated from content concerns, and that interactive complexity is encapsulated within well-defined boundaries. Each component follows the compound component pattern where appropriate, allowing flexible composition while maintaining visual consistency.

5.1 Layout Component Tree

Component	Type	Animation	Responsibility
RootLayout	Server	None	Font loading, theme provider, metadata
PageTransition	Client	Framer Motion	Route transition fade + slide
Header	Client	GSAP ScrollTrigger	Scroll-responsive header, hide/show on scroll
Navigation	Client	Framer Motion	Desktop mega-menu, hover animations
MobileMenu	Client	Framer Motion	Full-screen overlay, staggered link entrance
Footer	Server	None	Contact info, social links, sitemap

Table 6: Layout Component Hierarchy

5.2 Section Component Tree

Component	Animation	Key Props	Performance Notes
HeroSection	GSAP timeline	videoSrc, title, subtitle	Video lazy-loaded, poster fallback
BridalShowcase	Framer carousel	images, descriptions	Intersection-triggered, off-screen paused
ServicesGrid	Staggered reveal	services[], category	Virtualized for large catalogs
GalleryMasonry	Scroll + filter	images[], categories	Infinite scroll, progressive load

TestimonialsCarousel	Autoplay + swipe	testimonials[]	Reduced motion: static grid fallback
CTABanner	Parallax scroll	title, ctaLink	Background video throttled to 15fps
VIPSection	Card hover effects	tiers[], benefits	SVG decorations, no heavy animation

Table 7: Section Component Hierarchy

5.3 Booking Component Tree

The booking system is implemented as a multi-step wizard with a finite state machine governing transitions between steps. The `BookingWizard` component orchestrates four child steps: `ServiceSelector`, `StylistSelector`, `DateTimePicker`, and `BookingConfirmation`. Each step validates its own state before allowing forward navigation, and the entire flow is managed by a Zustand store that persists partial bookings to `localStorage`, allowing users to resume incomplete bookings on return visits. The `DateTimePicker` integrates with Firebase Realtime Database to display available slots in real-time, with optimistic UI updates when a slot is selected.

6. Animation Strategy

The animation strategy is the single most critical differentiator between a standard salon website and a luxury experience platform. Animation is not decoration; it is communication. Every animation serves one of three purposes: **atmosphere** (establishing the luxury feel), **feedback** (confirming user actions), or **navigation** (guiding attention and reducing cognitive load). Animations that serve none of these purposes are removed. The strategy is divided into three tiers of animation intensity, with reduced-motion support at every level.

6.1 Animation Technology Stack

Technology	Use Case	Performance Profile
Framer Motion	Page transitions, layout animations, gesture-driven interactions, presence animations	GPU-accelerated, requestAnimationFrame-based
GSAP + ScrollTrigger	Scroll-driven animations, parallax, timeline sequencing, text reveals	GPU-accelerated, ScrollTrigger optimized
CSS Animations	Hover states, micro-interactions, shimmer effects, pulse animations	Compositor-only, 60fps guaranteed
View Transitions API	Route-to-route shared element transitions (future enhancement)	Native browser optimization

Table 8: Animation Technology Stack

6.2 Animation Tier System

Tier 1 - Atmospheric (Always Running): These animations create the luxury atmosphere and run continuously. They include the cursor glow effect (a soft champagne gold radial gradient that follows the mouse on desktop), the hero section parallax depth effect (background video scrolls at 0.5x speed), and the shimmer effect on gold borders and accents. All Tier 1 animations use CSS transforms and opacity only, ensuring they run on the compositor thread without triggering layout or paint cycles. On mobile, the cursor glow is disabled and parallax is reduced to a simple translate with reduced motion media query support.

Tier 2 - Reveal (Scroll-Triggered): These animations bring content into view as the user scrolls, creating a sense of cinematic progression. Sections fade up with a slight Y-axis translation (20px to 0) over 0.8 seconds with a cubic-bezier(0.16, 1, 0.3, 1) easing. Cards within grids stagger their entrance with 80ms delay between items. Images scale from 1.05 to 1.0 as they enter the viewport, creating a subtle zoom-out effect. All Tier 2 animations use GSAP ScrollTrigger with scrub mode disabled (trigger-only) for discrete entrance animations, and scrub mode enabled for continuous parallax effects. Elements are lazy-initialized, meaning GSAP instances are only created when the element approaches the viewport.

Tier 3 - Interactive (User-Triggered): These animations respond to direct user input. Buttons feature a magnetic hover effect where the button subtly follows the cursor within a 40px radius. The mobile menu opens with a staggered animation: the overlay fades in, then navigation links slide in from the right with 50ms stagger. The booking wizard transitions between steps with a shared-layout animation that morphs the step indicator. Gallery lightbox opens with a spring animation (stiffness: 300, damping: 30) that scales the image from its thumbnail position to the full-screen view.

6.3 Performance Budget

Metric	Budget	Measurement Method
First Contentful Paint	< 1.2s	Lighthouse mobile 4G
Largest Contentful Paint	< 2.0s	Lighthouse mobile 4G
Cumulative Layout Shift	< 0.05	Chrome UX Report
Total Animation JS	< 45KB gzipped	Bundle analyzer
Animation Frame Budget	< 16ms per frame	Chrome DevTools Performance
GSAP Bundle (tree-shaken)	< 25KB gzipped	Custom build with ScrollTrigger only
Framer Motion Bundle	< 18KB gzipped	Lazy-loaded per route

Table 9: Animation Performance Budget

7. Responsive Design Strategy

The responsive design strategy is **mobile-first by architecture**, not by afterthought. Every component is designed for a 375px viewport first, then progressively enhanced for larger screens. This is critical because over 75% of salon discovery traffic in Pakistan comes from mobile devices, primarily on 4G connections with varying signal strength. The design system uses a fluid typography scale based on clamp() functions, container queries for component-level responsiveness, and CSS Grid with named areas for layout reconfiguration at each breakpoint.

7.1 Breakpoint System

Breakpoint	Width	Layout Strategy	Key Adaptations
xs (mobile)	375-639px	Single column, stacked sections	Full-screen mobile menu, touch-optimized targets
sm (large mobile)	640-767px	Single column with wider gutters	2-column service cards, larger hero text
md (tablet)	768-1023px	2-column grid, sidebar available	Split booking layout, gallery 2-col masonry
lg (desktop)	1024-1279px	3-column grid, hover states enabled	Full navigation bar, cursor glow active
xl (large desktop)	1280px+	3-4 column grid, max-width container	Cinematic hero scale, enhanced parallax

Table 10: Responsive Breakpoint System

7.2 Fluid Typography Scale

Typography uses CSS clamp() for fluid sizing, eliminating discrete font-size jumps at breakpoints while maintaining readability across all viewport sizes. The scale is defined with a minimum size (mobile), preferred size (vw-based), and maximum size (desktop cap). Headings use Playfair Display at sizes ranging from clamp(28px, 5vw, 56px) for H1 to clamp(16px, 2.5vw, 20px) for H3. Body text uses Inter at clamp(14px, 1.5vw, 16px) for primary content and Cormorant Garamond at clamp(15px, 1.6vw, 18px) for editorial-style descriptions. Line heights scale proportionally to maintain readability at each size, with CJK text receiving an additional 0.1 line-height multiplier to prevent character crowding.

7.3 Touch-First Interaction Design

All interactive elements are designed for touch-first interaction with progressive enhancement for pointer devices. Touch targets maintain a minimum 44x44px hit area as per WCAG 2.5.5 guidelines, with 48px preferred for primary actions. Swipe gestures are implemented for the gallery carousel (using Framer Motion drag constraints) and the testimonial slider. The booking wizard supports swipe-to-advance between

steps on mobile. Hover effects are desktop-only, activated via the `@media (hover: hover)` media query, ensuring no hover-dependent functionality on touch devices. The magnetic button effect and cursor glow are entirely disabled on touch devices, replaced by tap-scale feedback animations (scale: 0.97 on active state) that provide equivalent tactile feedback.

8. State Management Approach

State management is divided into three domains, each managed by the most appropriate tool for its characteristics: server state (API data, CMS content) is managed by TanStack Query, client UI state (menu open/close, active filters) is managed by Zustand, and form state is managed by React Hook Form with Zod validation. This tripartite approach avoids the common pitfall of using a single state management solution for all state types, which leads to either over-fetching (Zustand for server state) or under-caching (TanStack Query for UI state).

8.1 Zustand Stores

Zustand manages two primary stores. The **bookingStore** manages the multi-step booking flow state machine, including selected service, preferred stylist, chosen date/time, and customer details. It implements a finite state machine with explicit transitions, ensuring that users cannot skip steps or reach invalid states. The store also persists partial bookings to `localStorage` with a 24-hour TTL, allowing users to close the browser and resume their booking later. The **uiStore** manages transient UI state: mobile menu visibility, active gallery filter, lightbox state, and header scroll position. This store is intentionally ephemeral and is not persisted, as UI state should reset on page reload to provide a fresh experience.

8.2 TanStack Query Configuration

Query Key	Data Source	Stale Time	Cache Time	Refetch Strategy
["services"]	Sanity CMS	5 minutes	30 minutes	On window focus
["services", slug]	Sanity CMS	5 minutes	30 minutes	On window focus
["gallery"]	Sanity CMS	10 minutes	60 minutes	On mount
["testimonials"]	Sanity CMS	15 minutes	60 minutes	On mount
["availability", date]	Firebase RTDB	0 (real-time)	5 minutes	Real-time listener
["bookings"]	Prisma API	1 minute	10 minutes	On window focus
["vip-tiers"]	Sanity CMS	30 minutes	120 minutes	On mount

Table 11: TanStack Query Configuration

8.3 Server Actions

Next.js Server Actions handle all mutation operations: booking creation, contact form submission, and testimonial submission. Each action validates input with Zod schemas before executing, returns typed responses with success/error states, and implements optimistic updates on the client via TanStack Query's `useMutation` hook. The booking creation action uses a Firebase transaction to atomically check slot availability and create the booking, preventing race conditions during concurrent booking attempts. Error handling follows a structured pattern: validation errors return field-level messages for form display, server errors return a generic message with a correlation ID for debugging, and network errors trigger a retry with exponential backoff.

9. Firebase Architecture

Firebase serves three primary functions in the architecture: **real-time booking availability synchronization**, **push notification delivery**, and **analytics event collection**. The Firebase Realtime Database is used specifically for the availability system because it provides sub-100ms synchronization across clients, which is essential during bridal season when multiple users may be attempting to book the same time slot simultaneously. Firebase Cloud Messaging handles booking confirmation and reminder notifications. Firebase Analytics provides granular event tracking that feeds into the admin dashboard.

9.1 Realtime Database Schema

The Firebase Realtime Database uses a denormalized schema optimized for read performance, since availability checks are far more frequent than writes. The schema is organized by date, with each date containing time slots and each slot containing availability status and booking reference. This flat structure allows a client to fetch availability for a specific date with a single read operation, without needing to traverse nested structures or filter on the client.

```
availability/{date}/slots/{time}: { status: "available" | "booked" | "blocked", bookingId: string | null, updatedAt: number }
```

```
bookings/{bookingId}: { id, serviceId, stylistId, date, time, customerName, customerPhone, status, createdAt }
```

```
stylists/{stylistId}: { id, name, specialties[], availability/{date}/slots/{time} }
```

9.2 Booking Flow with Firebase Transactions

The booking creation flow uses a Firebase multi-path transaction to ensure atomicity. When a user selects a time slot, the client sends a booking request to the Next.js API route. The server-side handler initiates a Firebase transaction that: (1) reads the current availability status of the selected slot, (2) verifies it is still "available", (3) writes the booking record to `/bookings/{bookingId}`, (4) updates the slot status to "booked" with a reference to the booking, and (5) commits the transaction atomically. If the slot was booked by another user between the read and write, the transaction fails and the server returns a conflict error,

prompting the client to refresh availability and select a different slot. This approach eliminates double-booking without requiring pessimistic locks or queue-based systems.

9.3 Security Rules

Firebase security rules enforce that unauthenticated users can only read availability data, while booking writes must come through the Next.js API route (authenticated via a service account). Direct client writes to the database are prohibited, ensuring all booking mutations pass through validation logic. Admin users can read all bookings and update booking statuses, but cannot directly modify availability slots (which are managed by the transaction system). Rate limiting is implemented at the API route level using a token bucket algorithm, allowing 10 availability reads per minute and 3 booking attempts per minute per IP address.

10. Sanity CMS Schema Plan

Sanity CMS serves as the headless content management system for all editorial content: services, testimonials, gallery items, bridal showcase entries, VIP tier definitions, and site-wide settings. The CMS is configured with Sanity Studio embedded at the /admin/content route, providing content editors with a familiar authoring environment without leaving the platform. Real-time preview is enabled via Sanity's live content API, allowing editors to see changes reflected on the frontend within seconds using Next.js draft mode and on-demand ISR revalidation.

10.1 Schema Definitions

Schema	Fields	Relationships	Validation
service	title, slug, description, category, price, duration, image, featured, sortOrder	category -> serviceCategory	Required: title, slug, price, duration
serviceCategory	title, slug, description, icon, sortOrder	Has many: service	Required: title, slug
testimonial	name, rating, review, service, image, featured, date	service -> service	Rating: 1-5, review max 500 chars
galleryItem	title, image, category, tags[], featured, date	category -> galleryCategory	Image min 1200px width
galleryCategory	title, slug	Has many: galleryItem	Required: title, slug
bridalPackage	title, slug, description, images[], price, includes[], process[], featured	None (standalone)	Min 3 images, min 3 includes

<code>vipTier</code>	<code>title, slug, price, duration, benefits[], color, icon, featured</code>	None (standalone)	Min 5 benefits, required price
<code>siteSettings</code>	<code>siteName, tagline, heroVideo, heroImage, socialLinks[], contactInfo[], hours[], whatsappNumber</code>	Singleton document	Required: <code>siteName</code> , <code>whatsappNumber</code>

Table 12: Sanity CMS Schema Definitions

10.2 Content Workflow

The content workflow implements a draft-publish model with Sanity's built-in document actions. Content editors create documents in draft state, which are invisible to the public API but visible in preview mode via a secret draft URL. When a document is published, a Sanity webhook fires to the Next.js `/api/webhook/sanity` route, which triggers on-demand ISR revalidation for the affected page using `revalidatePath()`. This ensures that content changes propagate to the production site within 5 seconds without requiring a full rebuild. Deleted or unpublished documents trigger cache purging for the affected routes, ensuring stale content is never served.

10.3 Image Pipeline

All images are uploaded to Sanity and automatically synced to Cloudinary via a custom asset plugin. When an editor uploads an image in Sanity Studio, the plugin intercepts the upload, sends the original file to Cloudinary, and stores the Cloudinary public ID in the Sanity document. The frontend then uses Cloudinary URL transformations to generate responsive variants: automatic format negotiation (WebP for Chrome, AVIF for modern browsers, JPEG fallback), responsive width breakpoints (375w, 640w, 768w, 1024w, 1280w, 1920w), and quality optimization (`q_auto:good` for gallery images, `q_auto:eco` for thumbnails). This pipeline reduces image payload by 40-60% compared to serving unoptimized originals while maintaining visual quality indistinguishable from the source.

11. Deployment Structure

The deployment architecture is built on Vercel's edge-first platform, leveraging Incremental Static Regeneration, serverless functions, and edge middleware for optimal global performance. The deployment pipeline is fully automated via GitHub integration, with preview deployments for every pull request and production deployments on merge to the main branch. Environment-specific configurations are managed through Vercel's encrypted environment variables, with separate projects for production and preview environments.

11.1 Deployment Pipeline

Stage	Trigger	Actions	Environment
Preview	PR opened/updated	Build + deploy preview URL, run Lighthouse CI, comment PR with scores	Preview (non-production API keys)
Staging	Merge to develop branch	Full build + deploy to staging URL, run E2E tests with Playwright	Staging (production-like data)
Production	Merge to main branch	Full build + deploy to production, warm ISR cache, notify team	Production (live API keys)
Rollback	Manual trigger	Instant rollback to previous deployment via Vercel dashboard	Production

Table 13: Deployment Pipeline Stages

11.2 Environment Configuration

Variable	Production	Preview	Description
NEXT_PUBLIC_SANITY_PROJECT_ID	Production ID	Same	Sanity project identifier
SANITY_API_TOKEN	Production token	Same	Server-side Sanity API access
NEXT_PUBLIC_SANITY_DATASET	production	staging	Content dataset isolation
FIREBASE_PROJECT_ID	Production project	Same	Firebase project identifier
FIREBASE_SERVICE_ACCOUNT	Production key	Staging key	Server-side Firebase access
CLOUDINARY_CLOUD_NAME	Production cloud	Same	Cloudinary account identifier
CLOUDINARY_API_KEY	Production key	Same	Cloudinary API access
NEXTAUTH_SECRET	Production secret	Different	Admin auth encryption key
NEXT_PUBLIC_WHATSAPP_NUMBER	+92-XXX-XXXXXX X	Same	WhatsApp business number

Table 14: Environment Configuration

11.3 Performance Optimization Strategy

The deployment structure incorporates multiple performance optimization layers. At the CDN level, Vercel's edge network serves static assets from the nearest POP with aggressive cache headers (immutable for hashed assets, s-maxage=60 for ISR pages). At the application level, Next.js automatic code splitting

ensures each route loads only its required JavaScript, with shared dependencies extracted into a common chunk. Animation libraries are lazy-loaded: GSAP is loaded only after the hero section enters the viewport, and Framer Motion's AnimatePresence is dynamically imported for page transitions. At the image level, Cloudinary transformations combined with Next.js Image component's automatic srcset generation deliver the smallest appropriate image for each viewport and device pixel ratio.

12. Complete Development Roadmap

The development roadmap is organized into four phases over a 10-week timeline, with each phase producing a deployable increment that can be reviewed, tested, and iterated upon. The phasing strategy prioritizes the core luxury experience (hero, navigation, animations) in the earliest phases, ensuring that the emotional impact of the site is established before functional features (booking, admin) are layered on top. This approach allows stakeholders to experience and refine the luxury feel early, when changes are least expensive.

12.1 Phase 1: Foundation & Luxury Shell (Weeks 1-3)

Phase 1 establishes the visual foundation and luxury feel of the entire platform. This is the most critical phase because it defines the emotional baseline that all subsequent features must maintain. The primary deliverable is a fully animated homepage that demonstrates the luxury experience within 3 seconds of landing. Key milestones include: project scaffolding with Next.js 16, Tailwind CSS 4, and all configured tooling; brand design system implementation (colors, typography, spacing, component tokens); cinematic hero section with video/image parallax and animated text; animated navigation with scroll-responsive header and mobile menu; page transition system with Framer Motion; GSAP scroll-triggered reveal animations for all sections; and the footer with WhatsApp integration link. By the end of Phase 1, the homepage must pass the "3-second emotional impact test" when viewed on a mobile device over 4G.

Task	Duration	Dependencies	Deliverable
Project scaffolding & config	2 days	None	Running Next.js 16 with Tailwind, fonts, ESLint
Design system tokens & variables	2 days	Scaffolding	CSS custom properties, Tailwind config
Layout components (Header, Footer, Nav)	3 days	Design system	Animated header, responsive nav, footer
Hero section (video + parallax + text)	3 days	Layout	Cinematic hero with scroll animations
Section reveal animations (GSAP)	2 days	Hero	Scroll-triggered entrance animations

Page transition system	1 day	Layout	Framer Motion route transitions
Mobile menu & touch interactions	2 days	Layout	Full-screen mobile overlay, swipe gestures
Homepage sections (services preview, CTA)	2 days	Reveal animations	Complete homepage with all sections
Testing & performance optimization	2 days	All Phase 1 tasks	Lighthouse score > 90, < 2s LCP

Table 15: Phase 1 Task Breakdown

12.2 Phase 2: Content Pages & CMS (Weeks 4-5)

Phase 2 builds all content-driven pages and integrates Sanity CMS for dynamic content management. The Bridal Studio page receives the most attention, as it is the primary conversion page for the highest-value service. The gallery implements an Instagram-style masonry layout with category filtering and lazy loading. Key milestones include: Sanity project setup with all schema definitions; Sanity Studio integration at /admin/content; Bridal Studio page with cinematic showcase carousel; Services page with categorized grid and animated filtering; Gallery page with masonry layout, lightbox, and infinite scroll; About page with team section and brand story; Contact page with WhatsApp integration and map; and content population with production-quality images and copy. By the end of Phase 2, all public pages are live and editable through Sanity Studio.

12.3 Phase 3: Booking System & Admin (Weeks 6-8)

Phase 3 implements the revenue-generating features: the online booking system and the admin dashboard. The booking system is the most technically complex component, requiring Firebase real-time integration, optimistic UI updates, and race-condition prevention. The admin dashboard provides the salon staff with complete operational control. Key milestones include: Firebase project setup with security rules; multi-step booking wizard with real-time availability; booking API with Firebase transaction-based slot locking; WhatsApp confirmation message integration; admin dashboard with booking calendar and daily schedule; service management CRUD operations; gallery and testimonial moderation interfaces; and VIP membership enrollment flow. By the end of Phase 3, the platform is fully functional for both customers and staff, with end-to-end booking capability and administrative control.

12.4 Phase 4: Polish, Launch & Optimization (Weeks 9-10)

Phase 4 is dedicated to refinement, performance optimization, and launch preparation. This phase ensures that the luxury experience is consistent across all pages, all edge cases are handled, and the platform is production-ready. Key milestones include: cross-browser and cross-device testing (Chrome, Safari, Firefox, iOS Safari, Samsung Internet); performance audit and optimization (image pipeline tuning, bundle analysis, cache strategy refinement); accessibility audit (WCAG 2.1 AA compliance, screen reader testing, keyboard

navigation); SEO optimization (structured data, Open Graph, sitemap generation, robots.txt); analytics integration (Vercel Analytics, PostHog event tracking); content review with stakeholders and final copy editing; load testing for booking system under concurrent access; and production deployment with DNS configuration and SSL verification. By the end of Phase 4, the platform is launched and ready for public access with monitoring and alerting in place.

12.5 Post-Launch Roadmap

Following the initial launch, several enhancements are planned for the post-launch phase. These include: WhatsApp Business API integration for automated booking reminders and follow-up messages; an AI-powered style recommendation engine that suggests services based on customer history and preferences; a loyalty points system integrated with the VIP membership tiers; virtual try-on functionality using AR for bridal makeup previews; multi-language support (Urdu, English, Arabic) with next-intl internationalization; and a native mobile app using React Native that shares the booking and authentication logic with the web platform.

13. Technical Decisions & Trade-offs

13.1 Next.js 16 App Router vs. Pages Router

The App Router was selected over the Pages Router for three reasons: React Server Components reduce client-side JavaScript by 30-50% for content-heavy pages, the nested layout system eliminates redundant re-renders during navigation, and Server Actions provide a type-safe mutation layer without manual API route boilerplate. The trade-off is increased complexity in the caching model, which requires careful understanding of revalidation behaviors. This complexity is mitigated by the rendering strategy table (Section 2.3) that explicitly defines caching behavior for each route.

13.2 Firebase Realtime Database vs. Firestore

The Realtime Database was selected over Firestore for the booking availability system because it provides lower-latency synchronization (sub-100ms vs. 200-500ms for Firestore) and simpler atomic transactions for slot-locking operations. The flat, denormalized schema required by RTDB is a natural fit for the time-slot availability pattern, which is accessed by date key and does not require complex querying. Firestore would be preferred if the system needed to support multi-field queries (e.g., "find all available slots across all stylists for a given service"), but the current design avoids this by always querying availability for a specific date and stylist.

13.3 GSAP + Framer Motion (Dual Animation Library)

Using two animation libraries increases the JavaScript bundle size by approximately 43KB gzipped, but this trade-off is justified by the distinct strengths of each library. GSAP's ScrollTrigger provides scroll-driven animation precision that Framer Motion cannot match, including scrub-linked animations that progress in sync with scroll position. Framer Motion provides React-native layout animations and AnimatePresence for mount/unmount transitions that GSAP does not support declaratively. Both libraries are tree-shaken and lazy-loaded: GSAP loads after the hero section enters the viewport, and Framer Motion loads per-route. The combined cost is within the 45KB animation budget.

13.4 Cloudinary vs. Next.js Image Optimization

Cloudinary is used alongside Next.js Image for three reasons: automatic format negotiation (AVIF support before Vercel's built-in optimization), on-the-fly transformation pipelines (crop, overlay, background removal), and CMS-driven image management (Sanity-to-Cloudinary sync). The trade-off is an external dependency for image delivery, but Cloudinary's global CDN with 200+ POPs provides faster delivery than Vercel's image optimization in South Asian markets where the primary audience is located. A fallback to Next.js Image optimization is implemented for cases where Cloudinary is unavailable.

14. Quality Assurance Strategy

Quality assurance operates at three levels: automated (CI pipeline), semi-automated (periodic audits), and manual (stakeholder review). The CI pipeline runs on every pull request and includes TypeScript type checking, ESLint with Next.js rules, unit tests for business logic (booking state machine, availability calculations), and Lighthouse CI with performance budget enforcement (FCP < 1.2s, LCP < 2.0s, CLS < 0.05). Semi-automated audits run weekly and include full Lighthouse reports for all pages, bundle size tracking with size-limit, accessibility testing with axe-core, and visual regression testing with Playwright screenshot comparisons. Manual reviews occur at the end of each phase and involve stakeholder walkthroughs on multiple devices, content quality review against editorial standards, and emotional impact assessment against the "3-second test" benchmark.

14.1 Testing Architecture

Test Type	Tool	Coverage Target	Run Frequency
Unit Tests	Vitest	80% of business logic	Every PR
Component Tests	Testing Library + Vitest	Key interactive components	Every PR
Integration Tests	Playwright	Booking flow, admin CRUD	Every merge to develop
Visual Regression	Playwright screenshots	All pages + components	Weekly + before release
E2E Tests	Playwright	Critical user journeys	Every merge to main

Performance Tests	Lighthouse CI	All public pages	Every PR + weekly
Accessibility Tests	axe-core + Playwright	WCAG 2.1 AA compliance	Every PR + weekly
Load Tests	k6	Booking API under concurrency	Pre-launch + monthly

Table 16: Testing Architecture

15. SEO & Conversion Optimization

15.1 SEO Strategy

The SEO strategy targets high-intent local search queries in Lahore, including "best bridal salon Lahore," "luxury hair salon Lahore," and "bridal makeup Lahore." Technical SEO is handled at the framework level: Next.js generates static HTML for all content pages, the App Router provides automatic meta tag management via `generateMetadata()`, and JSON-LD structured data is injected for LocalBusiness, SalonOrSpa, and Service schemas. Dynamic sitemap generation via next-sitemap includes all service pages, gallery categories, and blog posts (future). Open Graph and Twitter Card meta tags ensure rich previews when pages are shared on social media, with carefully crafted OG images that feature the brand's luxury aesthetic. Page speed optimization (achieved through SSG, image optimization, and code splitting) directly contributes to Core Web Vitals scores, which are a confirmed Google ranking factor.

15.2 Conversion Optimization

The conversion optimization strategy is built around three principles: **friction reduction**, **trust building**, and **urgency creation**. Friction reduction is achieved by placing a "Book Now" CTA on every page (header, section footers, floating mobile button), implementing a one-click WhatsApp booking fallback for users who prefer messaging, and auto-filling customer details from `localStorage` on return visits. Trust building uses real testimonials with customer photos, before/after bridal transformations in the gallery, and certifications/awards prominently displayed on the About page. Urgency creation shows real-time availability counts ("Only 2 slots remaining this week") on the booking page and features seasonal bridal packages with countdown timers. A/B testing infrastructure is planned for post-launch, with PostHog feature flags enabling controlled rollout of conversion experiments.